

Binary Exponentiation

快速幂

定义

快速幂，二进制取幂（Binary Exponentiation，也称平方法），是一个在 $O(\log n)$ 的时间内计算 a^n 的小技巧，而暴力的计算需要 $O(n)$ 的时间。

这个技巧也常常用在非计算的场景，因为它可以应用在任何具有结合律的运算中。其中显然的是它可以应用于模意义下取幂、矩阵幂等运算，我们接下来会讨论。

解释

计算 a^n 的 n 次方表示将 a 个 a 乘在一起： $a \times a \times \dots \times a$ 个。然而当 n 太大的时候，这种方法就不太适用了。不过我们知道： $a^2 = a \times a$ 。二进制取幂的想法是，我们将取幂的任务按照指数的 **二进制表示** 来分割成更小的任务。

过程

迭代版本

首先我们将 n 表示为 2 进制，举一个例子：

因为 n 有 $\log_2 n$ 个二进制位，因此当我们知道了 n 的二进制表示后，我们只用计算 $\log_2 n$ 次乘法就可以计算出 a^n 。

于是我们只需要知道一个快速的方法来计算上述 3 的 n 次幂的序列。这个问题很简单，因为序列中（除第一个）任意一个元素就是其前一个元素的平方。举一个例子：

因此为了计算 a^n ，我们只需要将对应二进制位为 1 的整系数幂乘起来就行了：

将上述过程说得形式化一些，如果把 n 写作二进制为 $b_k b_{k-1} \dots b_1 b_0$ ，那么有：

其中 $a_i = a^{2^i}$ 。那么就有

根据上式我们发现，原问题被我们转化成了形式相同的子问题的乘积，并且我们可以在常数时间内从 a_{i-1} 推出 a_i 。

这个算法的复杂度是 $O(\log n)$ 的，我们计算了 $\log_2 n$ 个 a 次幂的数，然后花费 $O(\log n)$ 的时间选择二进制为 1 对应的幂来相乘。

递归版本

上述迭代版本中，由于 a_i 依赖于 a_{i-1} ，使得其转换为递归版本比较困难（一方面需要返回一个额外的 a_i ，对函数来说无法实现一个只返回计算结果的接口；另一方面则是必须从低位往高位计算，即从高位往低位调用，这也造成了递归实现的

困扰)，下面则提供递归版本的思路。

给定形式，即表示将的前位二进制位当作一个二进制数，则有如下变换：

那么有：

如上所述，在递归时，对于不同的递归深度是相同的处理：，即将当前递归的二进制数拆成两部分：最低位在递归出来时乘上去，其余部分则变成新的二进制数递归进入更深一层作相同的处理。

可以观察到，每递归深入一层则二进制位减少一位，所以该算法的时间复杂度也为。

实现

首先我们可以直接按照上述递归方法实现：



C++Python

```
1 long long bnpow(long long a, long long b) {
2     if (b == 0) return 1;
3     long long res = bnpow(a, b / 2);
4     if (b % 2)
5         return res * res * a;
6     else
7         return res * res;
8 }
```

```
1 def bnpow(a, b):
2     if b == 0:
3         return 1
4     res = bnpow(a, b // 2)
5     if (b % 2) == 1:
6         return res * res * a
7     else:
8         return res * res
```

第二种实现方法是非递归式的。它在循环的过程中将二进制位为1时对应的幂累乘到答案中。尽管两者的理论复杂度是相同的，但第二种在实践过程中的速度是比第一种更快的，因为递归会花费一定的开销。



C++Python

```
1 long long bnpow(long long a, long long b) {
2     long long res = 1;
3     while (b > 0) {
4         if (b & 1) res = res * a;
5         a = a * a;
6         b >>= 1;
7     }
8     return res;
9 }
```

```

1 def binpow(a, b):
2     res = 1
3     while b > 0:
4         if b & 1:
5             res = res * a
6         a = a * a
7         b >>= 1
8     return res

```

模板: [Luogu P1226](#)

应用

模意义下取幂

▼ 问题描述

计算。

这是一个非常常见的应用，例如它可以用于计算模意义下的乘法逆元。

既然我们知道取模的运算不会干涉乘法运算，因此我们只需要在计算的过程中取模即可。



C++Python

```

1 long long binpow(long long a, long long b, long long m) {
2     a %= m;
3     long long res = 1;
4     while (b > 0) {
5         if (b & 1) res = res * a % m;
6         a = a * a % m;
7         b >>= 1;
8     }
9     return res;
10 }

```

```

1 def binpow(a, b, m):
2     a = a % m
3     res = 1
4     while b > 0:
5         if b & 1:
6             res = res * a % m
7         a = a * a % m
8         b >>= 1
9     return res

```

注意：根据费马小定理，如果 p 是一个质数，我们可以计算 a^{p-2} 来加速算法过程。

计算斐波那契数

▼ 问题描述

计算斐波那契数列第 n 项。

根据斐波那契数列的递推式，我们可以构建一个 2×2 的矩阵来表示从 $n-1$ 到 n 的变换。于是在计算这个矩阵的 n 次幂的时候，我们使用快速幂的思想，可以在 $\log n$ 的时间内计算出结果。对于更多的细节参见 [斐波那契数列](#)，矩阵快速幂的实现参见 [矩阵加速递推](#) 中的实现。

多次置换

▼ 问题描述

给你一个长度为 n 的序列和一个置换，把这个序列置换 k 次。

简单地把这个置换取 k 次幂，然后把它应用到序列上即可。时间复杂度是 $O(n)$ 的。

注意：给这个置换建图，然后在每一个环上分别做 k 次幂（事实上做一下对环长取模的运算即可）可以取得更高效的算法，达到 $O(n)$ 的复杂度。

加速几何中对点集的操作

引入

三维空间中， n 个点，要求将 k 个操作都应用于这些点。包含 3 种操作：

1. 沿某个向量移动点的位置（Shift）。
2. 按比例缩放这个点的坐标（Scale）。
3. 绕某个坐标轴旋转（Rotate）。

还有一个特殊的操作，就是将一个操作序列重复 k 次（Loop），这个序列中也可能有 Loop 操作（Loop 操作可以嵌套）。现在要求你在低于 T 的时间内将这些变换应用到这个 n 个点，其中 T 表示把所有的 Loop 操作展开后的操作序列的长度。

解释

让我们来观察一下这三种操作对坐标的影响：

1. Shift 操作：将每一维的坐标分别加上一个常量；
2. Scale 操作：把每一维坐标分别乘上一个常量；
3. Rotate 操作：这个有点复杂，我们打算深入探究，不过我们仍然可以使用一个线性组合来表示新的坐标。

可以看到，每一个变换可以被表示为对坐标的线性运算，因此，一个变换可以用一个 4×4 的矩阵来表示：

使用这个矩阵就可以将一个坐标（向量）进行变换，得到新的坐标（向量）：

你可能会问，为什么一个三维坐标会多一个 1 出来？原因在于，如果没有这个多出来的 1，我们没法使用矩阵的线性变换来描述 Shift 操作。

过程

接下来举一些简单的例子来说明我们的思路：

1. Shift 操作：让 x 坐标方向的位移为 a ， y 坐标的位移为 b ， z 坐标的位移为 c ：
2. Scale 操作：把 x 坐标拉伸 10 倍， y 坐标拉伸 5 倍：
3. Rotate 操作：绕 z 轴旋转 θ 弧度，遵循右手定则（逆时针方向）

现在，每一种操作都被表示为了一个矩阵，变换序列可以用矩阵的乘积来表示，而一个 Loop 操作相当于取一个矩阵的 k 次幂。这样可以用 计算出整个变换序列最终形成的矩阵。最后将它应用到 n 个点上，总复杂度 $O(n^3 \log k)$ 。

定长路径计数

▼ 问题描述

给一个有向图（边权为 1），求任意两点 u 间从 u 到 v ，长度为 k 的路径的条数。

我们把该图的邻接矩阵 M 取 k 次幂，那么 M^k 就表示从 u 到 v 长度为 k 的路径的数目。该算法的复杂度是 $O(n^3 \log k)$ 。有关该算法的细节请参见 [矩阵](#) 页面。

模意义下大整数乘法

计算 $a \cdot b \pmod m$ 。

与二进制取幂的思想一样，这次我们将其中的一个乘数表示为若干个 2 的整数次幂的和的形式。因为在对一个数做乘 2 并取模的运算的时候，我们可以转化为加减操作防止溢出。这样仍可以在 $O(\log a)$ 的时内解决问题。递归方法如下：

快速乘

但是 `__int128` 的「龟速乘」还是太慢了，这在很多对常数要求比较高的算法比如 Miller_Rabin 和 Pollard-Rho 中，就显得不够用了。所以我们要介绍一种可以处理模数在 `long long` 范围内、不需要使用黑科技 `__int128` 的、复杂度为 $O(\log a)$ 的「快速乘」。

我们发现：

我们巧妙运用 `unsigned long long` 的自然溢出：

于是在算出 $a \cdot b$ 后，两边的乘法和中间的减法部分都可以使用 `unsigned long long` 直接计算，现在我们只需要解决如何计算 $a \cdot b \pmod m$ 。

我们考虑先使用 `long double` 算出 $a \cdot b$ 再乘上 m^{-1} 。

既然使用了 `long double`，就无疑会有精度误差。极端情况就是第一个有效数字（二进制下）在小数点后一位。在 x86-64 机器下，`long double` 将被解释成 80 位拓展小数（即符号为 1 位，指数为 15 位，尾数为 64 位），所以 `long double` 最多能精确表示的有效位数为 15。所以 $a \cdot b$ 最差从第 15 位开始出错，误差范围为 2^{-15} 。乘上 m^{-1} 这个 15 位整数，误差范围为 2^{-15} ，再加上 $a \cdot b$ 的误差范围为 2^{-15} ，取整后误差范围位 2^{-15} 。于是乘上 m 后，误差范围变成 2^{-15} ，我们需要判断这两种情况。

因为 $a \cdot b$ 在 `long long` 范围内，所以如果计算结果 $a \cdot b \pmod m$ 在 0 到 $m-1$ 时，直接返回，否则返回 $a \cdot b - m$ ，当然你也可以直接返回 $a \cdot b \pmod m$ 。

代码实现如下：

```
1 long long binmul(long long a, long long b, long long m) {
2     unsigned long long c =
3         (unsigned long long)a * b -
4         (unsigned long long)((long double)a / m * b + 0.5L) * m;
5     if (c < m) return c;
6     return c + m;
7 }
```

高精度快速幂

► 前置技能

▼ 例题【NOIP2003 普及组改编·麦森数】（[原题在此](#)）

题目大意：从文件中输入 (b, p) ，计算 b^p 的最后 100 位数字（用十进制高精度数表示），不足 100 位时高位补 0。

代码实现如下：

```
1 #include <string>
2 #include <iostream>
3 using namespace std;
4 int a[505], b[505], t[505], i, j;
5
6 void mult(int x[], int y[]) // 高精度乘法
7 {
8     memset(t, 0, sizeof(t));
9     for (i = 1; i <= x[0]; i++) {
10         for (j = 1; j <= y[0]; j++) {
11             if (i + j - 1 > 100) continue;
12             t[i + j - 1] += x[i] * y[j];
13             t[i + j] += t[i + j - 1] / 10;
14             t[i + j - 1] %= 10;
15             t[0] = i + j;
16         }
17     }
18     memcpy(b, t, sizeof(b));
19 }
20
21 void ksm(int p) // 快速幂
22 {
23     if (p == 1) {
24         memcpy(b, a, sizeof(b));
25         return;
26     }
27     ksm(p / 2); // (2^(p/2))^2=2^p
28     mult(b, b); // 对b平方
29     if (p % 2 == 1) mult(b, a);
30 }
31
32 int main() {
33     cin.tie(nullptr)->sync_with_stdio(false);
34     int p;
35     cin >> p;
36     a[0] = 1; // 记录a数组的位数
37     a[1] = 2; // 对2进行平方
38     b[0] = 1; // 记录b数组的位数
39     b[1] = 1; // 答案数组
40     ksm(p);
41     for (i = 100; i >= 1; i--) {
42         if (i == 1) {
43             cout << b[i] - 1 << '\n'; // 最后一位减1
44         } else
45             cout << b[i];
46     }
47 }
```

同一底数与同一模数的预处理快速幂

在同一底数与同一模数的条件下，可以利用分块思想，用一定的时间（一般是 $\sqrt{p}$ ）预处理后用 $\sqrt{p}$ 的时间回答一次幂询问。

过程

1. 选定一个数，预处理出 到 与 到 的值并存在一个（或两个）数组里；
2. 对于每一次询问，将 拆分成，则，可以 求出答案。

关于这个数 的选择，我们一般选择 或者一个大小适当的 的次幂（选择 可以使预处理较优，选择 的次幂可以使用位运算优化/简化计算）。

► 参考代码

习题

- [UVa 1230 - MODEX](#)
- [UVa 374 - Big Mod](#)
- [UVa 11029 - Leading and Trailing](#)
- [Codeforces - Parking Lot](#)
- [SPOJ - The last digit](#)
- [SPOJ - Locker](#)
- [LA - 3722 Jewel-eating Monsters](#)
- [SPOJ - Just add it](#)

本页面部分内容译自博文 [Бинарное возведение в степень](#) 与其英文翻译版 [Binary Exponentiation](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

参考资料与注释

1. 参见 [C 语言小数表示法 - 维基百科](#) ↩

本页面最近更新：2024/5/8 20:33:33, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [Aquistcev](#), [billchenchina](#), [CBW2007](#), [CCXXI](#), [chinggg](#), [Enter-tainer](#), [eyedeng](#), [FFjet](#), [Great-designer](#), [H-J-Granger](#), [Henry-ZHR](#), [hsfzLZH1](#), [Hszzzx](#), [lr1d](#), [Jude Gao](#), [kenlig](#), [kfy666](#), [Konano](#), [ksyx](#), [luoguyuntianming](#), [Marcythm](#), [Menci](#), [NachtgeistW](#), [ouuan](#), [Peanut-Tang](#), [qwqAutomaton](#), [shawlleyw](#), [shenshuaijie](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [TrisolarisHD](#), [TRSWNCA](#), [Xeonacid](#), [Zhangjiacheng2006](#), [Zhoier](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用